



## Implementación de la extensión Atomic de Risc-v en un procesador pipeline

### Integrantes:

| APELLIDOS, NOMBRES            | Participación |
|-------------------------------|---------------|
| Thiago Gabriel Rivera Matta   | 100%          |
| Jesús Valentín Niño Castañeda | 100%          |
| Reyes Medina, Jordinn Martin  | 100%          |

### ASIGNATURA:

Arquitectura de Computadoras

### DOCENTE:

Carlos Williams

## ÍNDICE

|                                                                                                      |           |
|------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Fundamento Teórico del Pipeline.....</b>                                                       | <b>3</b>  |
| <b>1.1. Concepto de Pipeline.....</b>                                                                | <b>3</b>  |
| <b>1.2. Datapath Pipeline.....</b>                                                                   | <b>3</b>  |
| <b>1.3. Unidades de Control Pipeline.....</b>                                                        | <b>3</b>  |
| <b>2. Implementación de las Instrucciones.....</b>                                                   | <b>1</b>  |
| 2.1. Instrucciones Implementadas.....                                                                | 1         |
| 2.2. Cambios realizados en el código.....                                                            | 2         |
| 2.3. Modificaciones en la unidad de control.....                                                     | 2         |
| 2.4. Modificaciones en el datapath.....                                                              | 2         |
| 2.5. Diagrama de el datapath final.....                                                              | 2         |
| <b>3. Programa (1): ISA sin Dependencias.....</b>                                                    | <b>3</b>  |
| 3.1. Descripción del programa.....                                                                   | 3         |
| 3.2. Archivo .mem utilizado para pruebas de instrucciones.....                                       | 3         |
| 3.3. Waveform de ejecución.....                                                                      | 5         |
| 3.4. Validación de resultados.....                                                                   | 5         |
| <b>4. Fundamento Teórico de la Hazard Unit.....</b>                                                  | <b>5</b>  |
| 4.1. Concepto de Hazard.....                                                                         | 5         |
| 4.2. Tipos de Hazard.....                                                                            | 5         |
| Para mantener el flujo correcto del programa, la Hazard Unit implementa técnicas de resolución:..... | 5         |
| 4.3. Forwarding (Anticipación).....                                                                  | 5         |
| 4.4. Stalling (Detención / Burbuja).....                                                             | 5         |
| 4.5. Flushing (Vaciado).....                                                                         | 6         |
| <b>5. Comportamiento sin Hazard Unit.....</b>                                                        | <b>6</b>  |
| 5.1. Programa (2): ISA Test Forwarding.....                                                          | 6         |
| 5.2. Programa (3): ISA Test Stalling.....                                                            | 6         |
| 5.3. Programa (4): ISA Test Flushing.....                                                            | 7         |
| 5.4. Evidencia del funcionamiento incorrecto.....                                                    | 8         |
| <b>6. Implementación de la Hazard Unit.....</b>                                                      | <b>8</b>  |
| 6.1. Cambios realizados en el código.....                                                            | 8         |
| 6.2. Diagrama de datapath final.....                                                                 | 8         |
| <b>7. Validación de la Hazard Unit.....</b>                                                          | <b>9</b>  |
| 7.1. Prueba de Forwarding.....                                                                       | 9         |
| 7.2. Prueba de Stalling.....                                                                         | 9         |
| 7.3. Prueba de Flushing.....                                                                         | 9         |
| 7.4. Waveforms de verificación.....                                                                  | 10        |
| 7.5. Comparación antes y después de la implementación.....                                           | 10        |
| <b>8. Conclusiones.....</b>                                                                          | <b>10</b> |
| <b>Bibliografía.....</b>                                                                             | <b>10</b> |

# 1. Fundamento Teórico del Pipeline

## 1.1. Concepto de Pipeline

El pipeline es una técnica de implementación microarquitectural con el objetivo de mejorar el rendimiento de un sistema digital. Divide el procesador de ciclo único en cinco etapas de segmentación consecutivas e independientes, permitiendo que cinco instrucciones se ejecuten simultáneamente, una en cada etapa. (Harris & Harris, 2020)

En lugar de esperar a que una sola instrucción complete todo su ciclo, tan pronto como una instrucción pasa a la siguiente etapa, el procesador inicia la ejecución de una nueva instrucción. No reduce el tiempo de latencia de una instrucción, pero incrementa el rendimiento global del sistema, por lo que incrementa su importancia, dado que los microprocesadores ejecutan millones de instrucciones por segundo. En una arquitectura Risc-v, el ciclo se divide en 5 etapas:

1. Fetch : Búsqueda de la instrucción en la memoria.
2. Decode: Decodificación y lectura en el banco de registros.
3. Execute: Ejecución en la ALU.
4. Memory: El procesador lee o escribe en la memoria de datos.
5. Writeback: El procesador escribe el resultado en el banco de registros.

## 1.2. Datapath Pipeline

El datapath de un procesador pipeline contiene los elementos de hardware necesarios para procesar la información (ALU, registros) interconectados de manera secuencial. El datapath pipeline se forma dividiendo el datapath de único ciclo en cinco etapas separadas por registros de pipeline entre cada una de las etapas. (Harris & Harris, 2020)

## 1.3. Unidades de Control Pipeline

En un procesador pipeline, la unidad de control ya no es capaz de activar señales para todo el ciclo del reloj, ya que en el mismo instante coexisten hasta 5 instrucciones distintas en el datapath. La unidad de control se ubica en la etapa de decode para generar las señales de control requeridas para esa instrucción específica examinando los campos opcode y funct.

Estas señales se transmiten a lo largo del pipeline junto con los datos a través de los registros de pipeline. A medida que la instrucción avanza, las señales correspondientes a las etapas de ejecución se van aplicando en el momento exacto.

## 2. Implementación de las Instrucciones

### 2.1. Instrucciones Implementadas

La implementación del pipeline incluye las instrucciones requeridas para ejecutar correctamente los programas de prueba y para validar el comportamiento del datapath y de la Hazard Unit. En el cuadro siguiente se resumen las instrucciones soportadas y su función principal.

| Instrucción       | Formato | Operación principal                | Uso en el proyecto                                           |
|-------------------|---------|------------------------------------|--------------------------------------------------------------|
| <code>lw</code>   | I-type  | Carga desde memoria                | Prueba de stall por dependencia de datos                     |
| <code>addi</code> | I-type  | Suma inmediata                     | Inicialización de registros y pruebas de datos               |
| <code>slli</code> | I-type  | Shift izquierdo lógico inmediato   | Verificación de operaciones de desplazamiento con inmediatos |
| <code>xori</code> | I-type  | Xor inmediato                      | Validación de operaciones lógicas con inmediatos             |
| <code>srlr</code> | I-type  | Shift derecho lógico inmediato     | Verificación de desplazamiento lógico hacia la derecha       |
| <code>srai</code> | I-type  | Shift derecho aritmético inmediato | Verificación del manejo correcto del bit de signo            |

| Instrucción | Formato | Operación principal      | Uso en el proyecto                                      |
|-------------|---------|--------------------------|---------------------------------------------------------|
| ori         | I-type  | Or inmediato             | Validación de operaciones lógicas con inmediatos        |
| andi        | I-type  | And inmediato            | Validación de operaciones lógicas con inmediatos        |
| add         | R-type  | Suma de registros        | Validación de forwarding y ejecución aritmética         |
| sub         | R-type  | Resta de registros       | Verificación de operaciones aritméticas y comparaciones |
| sll         | R-type  | Shift izquierdo lógico   | Verificación de desplazamiento usando registros         |
| xor         | R-type  | Xor entre registros      | Validación de operaciones lógicas entre registros       |
| srl         | R-type  | Shift derecho lógico     | Verificación de desplazamiento lógico usando registros  |
| sra         | R-type  | Shift derecho aritmético | Verificación de extensión de signo en desplazamientos   |

| Instrucción | Formato | Operación principal              | Uso en el proyecto                                |
|-------------|---------|----------------------------------|---------------------------------------------------|
| or          | R-type  | Or entre registros               | Validación de operaciones lógicas entre registros |
| and         | R-type  | And entre registros              | Validación de operaciones lógicas entre registros |
| lui         | U-type  | Cargar en el intermedio superior | Inicialización eficiente de constantes de 32 bits |
| beq         | B-type  | Comparación y salto condicional  | Soporte para control de flujo                     |
| bne         | B-type  | Salto si no es igual             | Verificación de comparaciones                     |
| blt         | B-type  | Salto si es menor que            | Verificación de comparaciones con signo           |
| bge         | B-type  | Salto si es mayor o igual que    | Verificación de comparaciones con signo           |
| sw          | S-type  | Almacenamiento a memoria         | Verificación final del resultado en la memoria    |
| jalr        | I-type  | Salto incondicional con registro | Verificación de saltos indirectos                 |

| Instrucción | Formato | Operación principal | Uso en el proyecto |
|-------------|---------|---------------------|--------------------|
| jal         | J-type  | Salto incondicional | Prueba de flushing |

## 2.2. Cambios realizados en el código

Para convertir el diseño monociclo en un procesador pipeline, fue necesario separar las etapas funcionales y propagar señales de control entre registros. Se mantuvo la lógica de control original, pero se incorporaron registros intermedios entre Fetch, Decode, Execute, Memory y Writeback. Además, se añadió la unidad de riesgos para gestionar dependencias de datos y saltos.

## 2.3. Modificaciones en la unidad de control

La unidad de control sigue decodificando el opcode y el campo `funct3`, pero ahora sus señales se almacenan en los registros del pipeline para que cada etapa opere con la instrucción correcta. En particular:

- `RegWrite`, `MemWrite`, `ALUSrc`, `ResultSrc` y `ALUControl` se propagan a través de las etapas.
- `ImmSrc` permite distinguir correctamente I, S, B y J-type.
- La señal `Branch` y la señal `Jump` se usan para actualizar el PC cuando corresponde.

## 2.4. Modificaciones en el datapath

El datapath se reorganizó en cinco etapas:

- **Fetch:** lectura de instrucción y cálculo de `PC + 4`.
- **Decode:** decodificación, lectura de registros y extensión del inmediato.
- **Execute:** cálculo de la ALU y decisión del salto/branch.
- **Memory:** acceso a la memoria de datos si la instrucción lo requiere.
- **Writeback:** escritura del resultado en el banco de registros.

Además, el datapath incorpora multiplexores de forwarding (`ForwardAE` y `ForwardBE`) y permite congelar o vaciar instrucciones mediante `StallF`, `StallD` y `FlushE`.

## 2.5. Diagrama de el datapath final

None

PC -> IMEM -> IF/ID -> ID/EX -> EX/MEM -> MEM/WB -> RegFile/ALU/MEM



None

Fetch: PC -> PCReg -> IMEM -> Instr

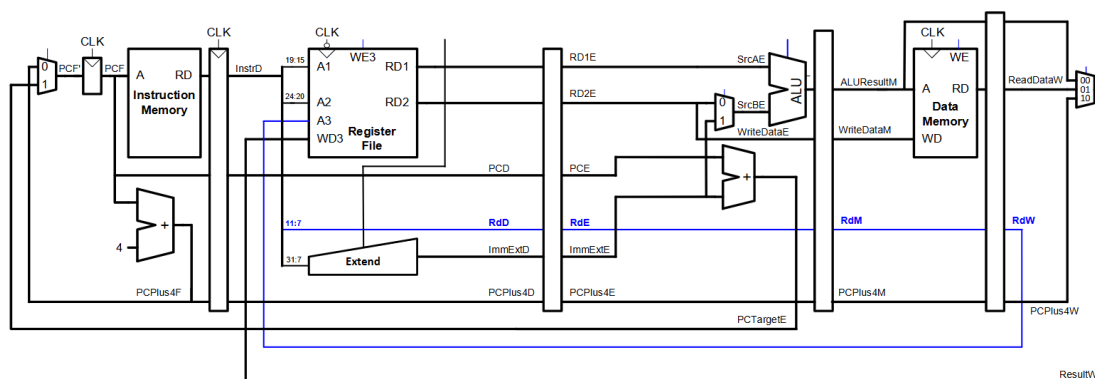
Decode: Instr -> Control + RegFile + Extend -> IF/ID

Execute: ID/EX -> ALU + Branch/JAL + Forwarding -> EX/MEM

Memory: EX/MEM -> DMem -> MEM/WB

Writeback: MEM/WB -> mux ResultSrc -> RegFile

Datapath de referencia:



### 3. Programa (1): ISA sin Dependencias

#### 3.1. Descripción del programa

Este programa verifica el funcionamiento básico del pipeline cuando no existen dependencias inmediatas entre instrucciones. Para evitar errores por lecturas prematuras, se utilizan instrucciones **nop** entre las operaciones que modifican y usan registros. El objetivo es comprobar que el procesador puede completar correctamente la secuencia de carga, suma y almacenamiento.



### 3.2. Archivo .mem utilizado para pruebas de instrucciones

La secuencia de instrucciones usada para esta prueba es la siguiente:

```
None
addi x2, x0, 5      # x2 = 5
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
addi x3, x0, 10     # x3 = 10
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
add  x4, x2, x3     # x4 = 15
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
addi x0, x0, 0      # NOP
sw   x4, 100(x0)    # mem[100] = 15
```

Contenido del archivo .mem:

```
None
00500113
00000013
00000013
00000013
00a00193
00000013
00000013
00000013
00310233
```

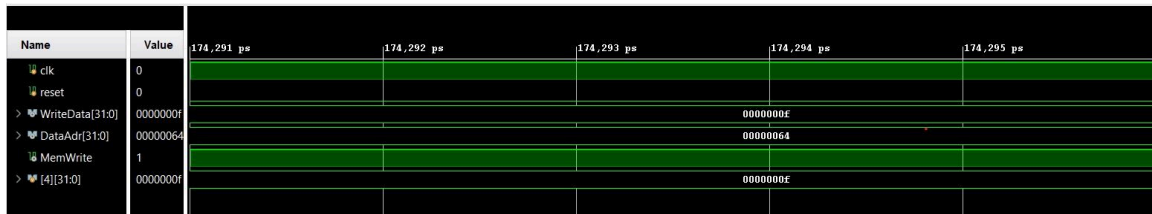
00000013

00000013

00000013

06402223

### 3.3. Waveform de ejecución



En el waveform se observa cómo el PC avanza secuencialmente, cómo `addi x2, x0, 5` y `addi x3, x0, 10` dejan los valores esperados en los registros, y cómo `add x4, x2, x3` genera el resultado 15. Finalmente, la señal de escritura en memoria activa la operación `sw` y se confirma que el dato que llega a la dirección 100 es correcto.

### 3.4. Validación de resultados

La validación se realiza verificando que durante la ejecución del `sw` el procesador escribe:

- `DataAdr = 100`
- `WriteData = 15`

Si ambas condiciones se cumplen, el testbench confirma el correcto funcionamiento del programa y termina con el mensaje de éxito.

## 4. Fundamento Teórico de la Hazard Unit

### 4.1. Concepto de Hazard

Un hazard (o riesgo) es una situación en la que una instrucción no puede ejecutarse en el ciclo correcto porque depende de recursos o datos que todavía no están disponibles. En un pipeline, estos riesgos aparecen con frecuencia cuando varias instrucciones avanzan simultáneamente.

### 4.2. Tipos de Hazard

- **Hazard estructural:** ocurre cuando dos instrucciones quieren usar el mismo recurso al mismo tiempo.
- **Hazard de datos:** aparece cuando una instrucción necesita un valor que otra instrucción aún no ha escrito.

- **Hazard de control:** ocurre con saltos o ramas, porque la siguiente instrucción no puede conocerse hasta evaluar la condición.

Para mantener el flujo correcto del programa, la Hazard Unit implementa técnicas de resolución:

#### 4.3. Forwarding (Anticipación)

El forwarding permite enviar el dato ya calculado desde una etapa posterior hacia la etapa que lo necesita, sin detener todo el pipeline. Esto evita esperar varios ciclos cuando una instrucción consume el resultado de otra que está en progreso.

#### 4.4. Stalling (Detención / Burbuja)

Cuando el dato no está disponible aún y no puede adelantarse, el pipeline detiene temporalmente al Fetch y al Decode. Esta pausa se conoce como stall o burbuja, y evita que una instrucción incorrecta avance en el datapath.

#### 4.5. Flushing (Vaciado)

El flushing se usa para eliminar la instrucción incorrecta que ya entró al pipeline cuando se detecta un salto o un branch que cambia el flujo del programa. Así se evita ejecutar instrucciones que no corresponden al camino correcto.

## 5. Comportamiento sin Hazard Unit

### 5.1. Programa (2): ISA Test Forwarding

Este programa muestra una dependencia entre `addi` y `add`. Sin forwarding, el `add` puede intentar usar datos todavía no actualizados en el banco de registros.

Instrucciones ensamblador (RISC-V):

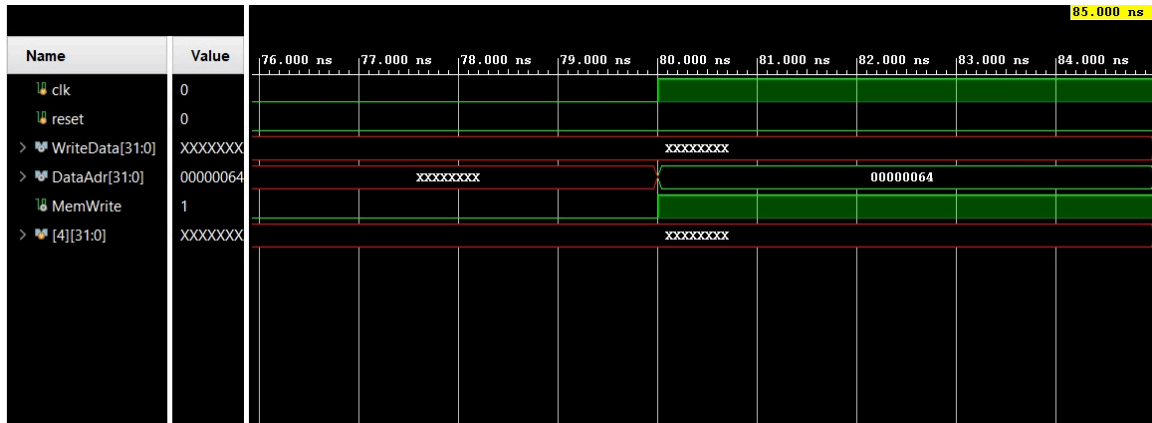
```
None
addi x2, x0, 5
addi x3, x0, 10
add x4, x2, x3
sw x4, 100(x0)
```

Contenido del archivo `.mem`:

```
None
00500113
00a00193
```

00310233

06402223



El resultado correcto debería ser 15 (F), sin embargo en WriteData podemos observar que está indefinido (X) denotando un error.

## 5.2. Programa (3): ISA Test Stalling

Este ejemplo valida el caso de una carga seguida de un uso inmediato del dato. Sin la detección del stall, la instrucción que necesita el valor cargado podría leer un dato viejo o incompleto.

Instrucciones ensamblador (RISC-V):

```
None
addi x2, x0, 15

sw    x2, 0(x0)

lw    x3, 0(x0)

addi x4, x3, 0

sw    x4, 100(x0)
```

Contenido del archivo `.mem`:

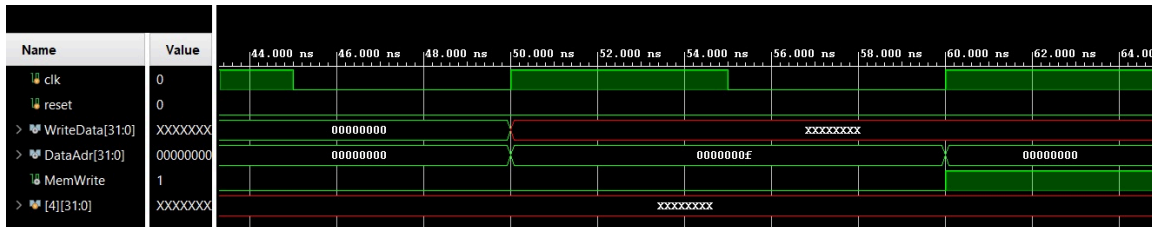
```
None
00f00113

00202023
```

00002183

00018213

06402223



Nuevamente el resultado esperado es 15(F), pero WriteData sale indefinido(X) demarcando un malfuncionamiento.

### 5.3. Programa (4): ISA Test Flushing

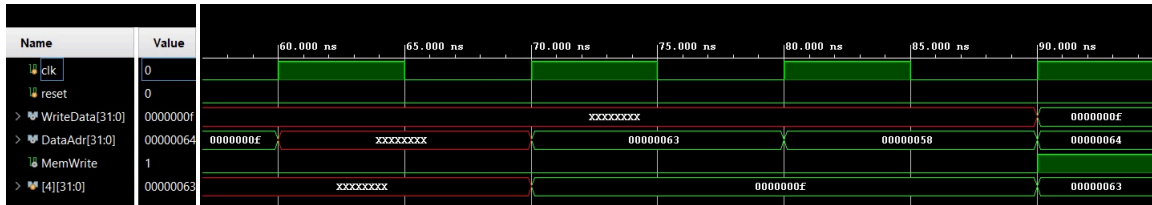
Este caso comprueba el comportamiento ante un salto. Sin el flushing, el pipeline podría ejecutar instrucciones que pertenecen al camino equivocado del programa.

Instrucciones ensamblador (RISC-V):

```
None
addi x4, x0, 15
jal x0, 12
addi x4, x0, 99
addi x4, x0, 88
sw x4, 100(x0)
```

Contenido del archivo **.mem**:

```
None
00f00213
00c0006f
06300213
05800213
06402223
```



En este caso el resultado es correcto 15(F), pero vemos como el valor de `x4` se altera ya que la instrucción posterior al `jal` persiste el pipeline.

## 6. Implementación de la Hazard Unit

Sin la Hazard Unit, los programas (2), (3) y (4) muestran errores porque el procesador intenta usar datos o instrucciones que aún no están listos. En los waveforms se aprecia que el valor almacenado en memoria no siempre coincide con el esperado, lo que confirma la necesidad de implementar forwarding, stalling y flushing para corregir el flujo del pipeline.

### 6.1. Cambios realizados en el código

La Hazard Unit se añadió a la conexión entre el datapath y el banco de registros. Su función principal es detectar:

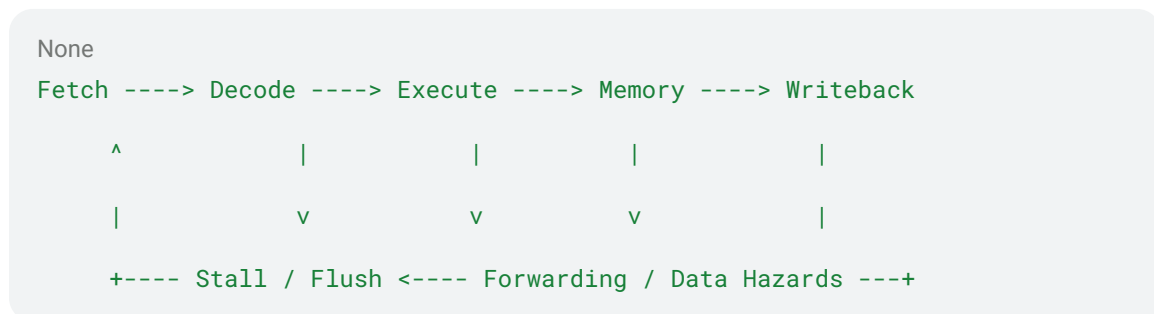
- dependencias de datos en la etapa `E`,
- necesidad de detener el pipeline para instrucciones `lw`,
- y saltos o branches que requieren vaciar instrucciones ya cargadas.

La lógica implementada en el código considera las señales `Rs1E`, `Rs2E`, `RdM`, `RdW`, `RegWriteM`, `RegWriteW`, `ResultSrcE0` y `PCSrcE` para generar:

- `ForwardAE` y `ForwardBE`,
- `StallF` y `StallD`,
- `FlushE`.

### 6.2. Diagrama de datapath final

El datapath final incluye la Hazard Unit como bloque de control adicional que monitorea las señales entre etapas y corrige el comportamiento del pipeline.



Datapath de referencia:



## 7.2. Prueba de Stalling

La prueba de stalling se centra en el caso **lw** seguido de un uso inmediato del dato cargado. El pipeline debe detenerse momentáneamente para que el dato llegue a la etapa correcta antes de continuar.



## 7.3. Prueba de Flushing

La prueba de flushing valida la corrección del salto **jal**. Cuando la instrucción de salto se detecta en Execute, la etapa siguiente debe vaciarse para que el flujo del programa continúe con la instrucción correcta.



## 7.4. Waveforms de verificación

Los waveforms muestran cómo, en cada caso:

- el PC no avanza indebidamente durante un stall,
- el valor correcto se propaga por el forwarding,
- y el pipeline elimina la instrucción equivocada cuando se aplica flushing.



### 7.5. Comparación antes y después de la implementación

Antes de la Hazard Unit, los programas (2), (3) y (4) presentan errores de lectura o de secuencia debido a dependencias. Después de la implementación, el pipeline mantiene la consistencia del flujo, evita lecturas incorrectas y escribe en memoria el valor esperado.

## 8. Conclusiones

El diseño del pipeline implementado en el proyecto permite ejecutar instrucciones RISC-V en cinco etapas y manejar correctamente los riesgos principales mediante forwarding, stalling y flushing. La validación con los programas de prueba demuestra que el procesador puede ejecutar una secuencia sin dependencias y corregir las dependencias de datos y de control cuando la Hazard Unit está activa. En conjunto, el sistema cumple con el objetivo de mostrar el funcionamiento correcto del pipeline y de evidenciar cómo la unidad de riesgos mejora la ejecución del programa.

## Bibliografía

- I. Harris, D., & Harris, S. (2020). *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann.